

ML HW2

1. Binary logistic regression

First, I want to talk about binary cross entropy, which is generated from the - log likelihood.

$$E(\mathbf{w}) = -\ln p(\mathbf{t}|\mathbf{X}, \mathbf{w})$$

So I define the loss function as below.

$$\text{Loss} = -\frac{1}{N} \sum_{i=1}^N y_i \cdot \log(p(y_i)) + (1 - y_i) \cdot \log(1 - p(y_i))$$

In the case $y = 1$, $\text{Loss} = -\log(p(y))$

If y approaches 1, loss approaches 0. On the other hand, as it approaches 0, the loss approaches infinity.

In the case $y = 0$, $\text{Loss} = -(1-y) \cdot \log(1-p(y))$

If y approaches 0, loss approaches 0. On the other hand, as it approaches 1, the loss approaches infinity.

$$\begin{aligned} L(w, b) &= -\frac{1}{n} \sum_{i=1}^n (y_i \log(\hat{y}_i) + (1-y_i) \log(1-\hat{y}_i)) \\ \hat{y}_i &= \frac{1}{1+e^{-z_i}}, \quad z_i = x_i^T w + b \\ \frac{d\hat{y}_i}{dz_i} &= \hat{y}_i(1-\hat{y}_i), \quad \frac{dL_i}{d\hat{y}_i} = -\left(\frac{y_i}{\hat{y}_i} - \frac{1-y_i}{1-\hat{y}_i}\right) \end{aligned}$$

$$\text{By chain rule} \rightarrow \frac{dL_i}{dz_i} = (\hat{y}_i - y_i)$$

$$\begin{aligned} \text{Since } \frac{dz_i}{dw} &= x_i^T, \quad \frac{dz_i}{db} = 1 \quad \xrightarrow{\text{Chain Rule}} \quad \frac{dL_i}{dw} = (\hat{y}_i - y_i) x_i \\ \frac{dL_i}{db} &= (\hat{y}_i - y_i). \end{aligned}$$

This defines dW and db

• Implementation

1. For the data : Normalization

```
X_train = X_train / 255.0
X_test = X_test / 255.0
```

2. Here I define the training process. I also use regularization to prevent overfitting.

```
def sigmoid(z):
    return 1 / (1 + np.exp(-z))

def train(X, y, epochs=1000, lr=0.01, lambda_l2=0.0):
    n_samples, n_features = X.shape
    W = np.zeros((n_features, 1))
    b = 0
    losses = []

    for epoch in tqdm(range(epochs), desc="Training"):
        z = np.dot(X, W) + b
        y_pred = sigmoid(z)

        error = y_pred - y

        dW = np.dot(X.T, error) / n_samples + lambda_l2 * W # 加上 L2 項的梯度
        db = np.sum(error) / n_samples

        W -= lr * dW
        b -= lr * db

        # Loss 也要加上 L2 懲罰項
        loss = -np.mean(y * np.log(y_pred + 1e-8) + (1 - y) * np.log(1 - y_pred + 1e-8))
        l2_penalty = (lambda_l2 / 2) * np.sum(W * W)
        total_loss = loss + l2_penalty
        losses.append(total_loss)

    return W, b, losses
```

3. Use cv to get the hyperparameter and check the validation score
Also I use a new function called train_with_validation, after each epoch of training, he will use the new w and b to calculate the validation loss to see if there is overfit.

```
def cross_validation(X, y, folds=5, epochs=1000, lr=0.1, lambda_l2=0.0):
    fold_size = len(X) // folds
    accuracies = []

    for fold in range(folds):
        val_start = fold * fold_size
        val_end = val_start + fold_size

        X_val = X[val_start:val_end]
        y_val = y[val_start:val_end]

        X_train_fold = np.vstack([X[:val_start], X[val_end:]])
        y_train_fold = np.vstack([y[:val_start], y[val_end:]])

        W, b, _ = train(X_train_fold, y_train_fold, epochs=epochs, lr=lr, lambda_l2=lambda_l2)
        y_val_pred = predict(X_val, W, b)

        acc = (y_val_pred == y_val).mean()
        accuracies.append(acc)

    return np.mean(accuracies)
```

As for the hyperparameter, I choose learning rate and lambda_l2 to optimize.

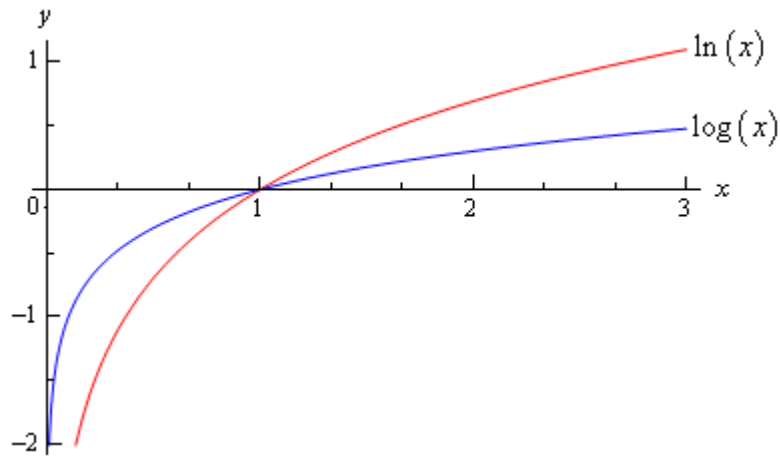
2. Multi-class classification

Let's talk about the multi cross entropy, which is also generated from the - log likelihood.

$$E(\mathbf{w}_1, \mathbf{w}_2, \dots, \mathbf{w}_K) = -\ln p(\mathbf{T}|\mathbf{X}, \mathbf{w}_1, \mathbf{w}_2, \dots, \mathbf{w}_K)$$

$$L = \frac{1}{N} \sum_i L_i = -\frac{1}{N} \sum_i \sum_{c=1}^M y_{ic} \log(p_{ic})$$

Where P_{ic} is the softmax function and y_{ic} is one-hot code, so if predicted correctly, you will get a smaller loss according to the below picture.



- Implementation

```

def one_hot(y, num_classes):
    return np.eye(num_classes)[y.reshape(-1)]

def train(X, y, epochs=1000, lr=0.1, lambda_l2=0.0, num_classes=10):
    n_samples, n_features = X.shape
    W = np.zeros((n_features, num_classes))
    b = np.zeros((1, num_classes))

    y_onehot = one_hot(y, num_classes)
    losses = []

    for epoch in tqdm(range(epochs), desc="Training"):
        z = np.dot(X, W) + b
        y_pred = softmax(z)

        error = y_pred - y_onehot

        dW = np.dot(X.T, error) / n_samples + lambda_l2 * W
        db = np.sum(error, axis=0, keepdims=True) / n_samples

        W -= lr * dW
        b -= lr * db

        loss = -np.mean(np.sum(y_onehot * np.log(y_pred + 1e-8), axis=1))
        l2_penalty = (lambda_l2 / 2) * np.sum(W * W)
        total_loss = loss + l2_penalty
        losses.append(total_loss)

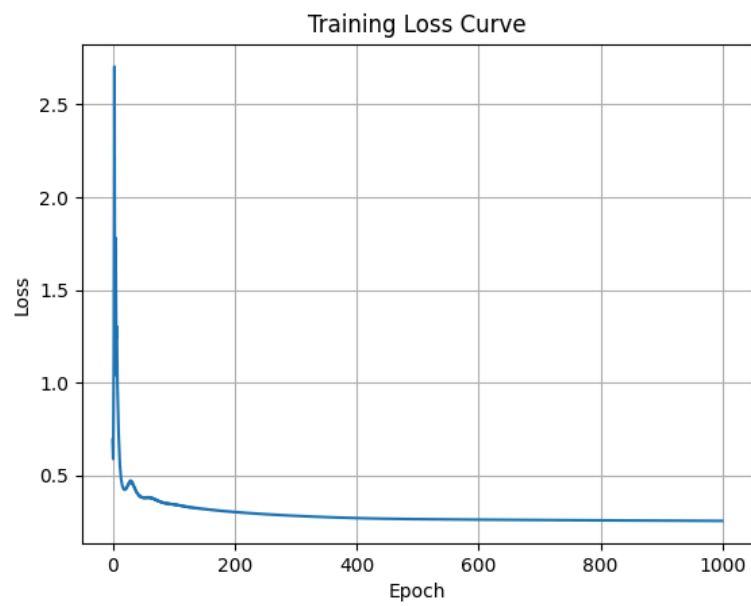
    return W, b, losses

```

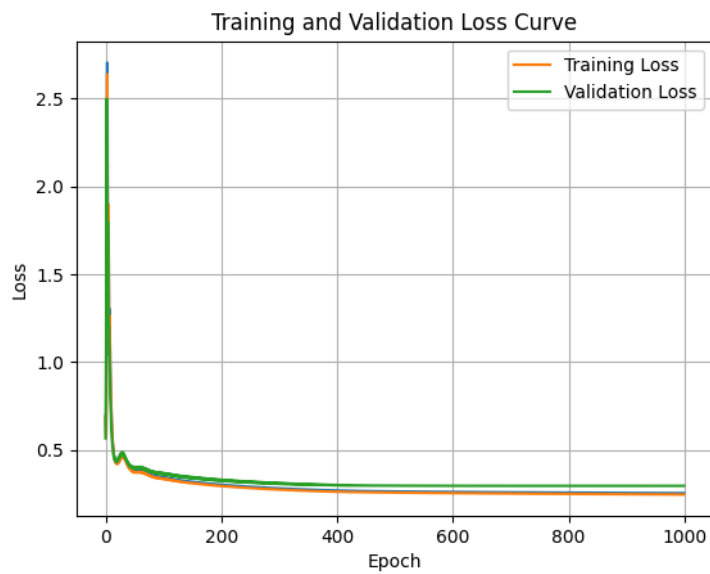
Just Change sigmoid to one_hot, others remain the same.

3. Result

1. Binary Classification

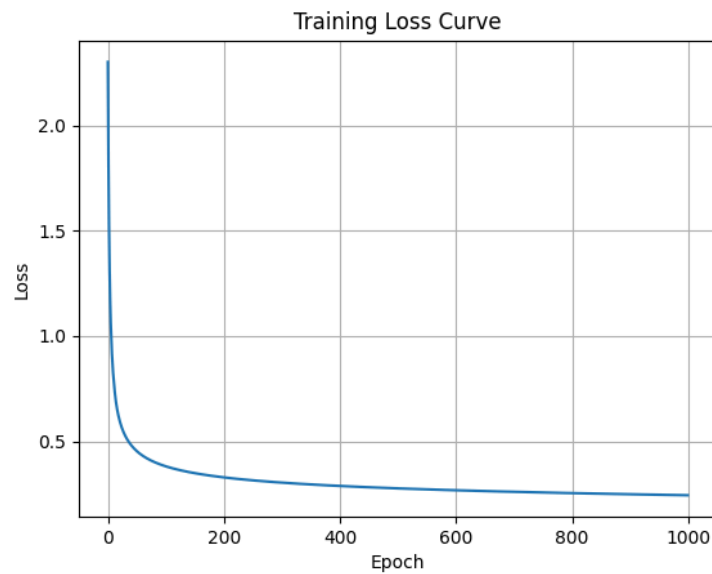


As you can see, the loss keeps decreasing without divergence.

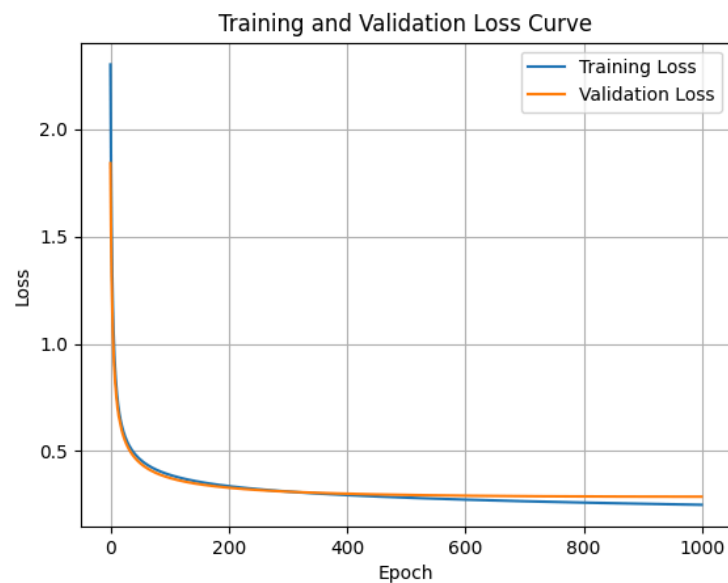


Use `train_with_validation` to check, it seems that there is no overfit problem.

2. Multiclass Classification



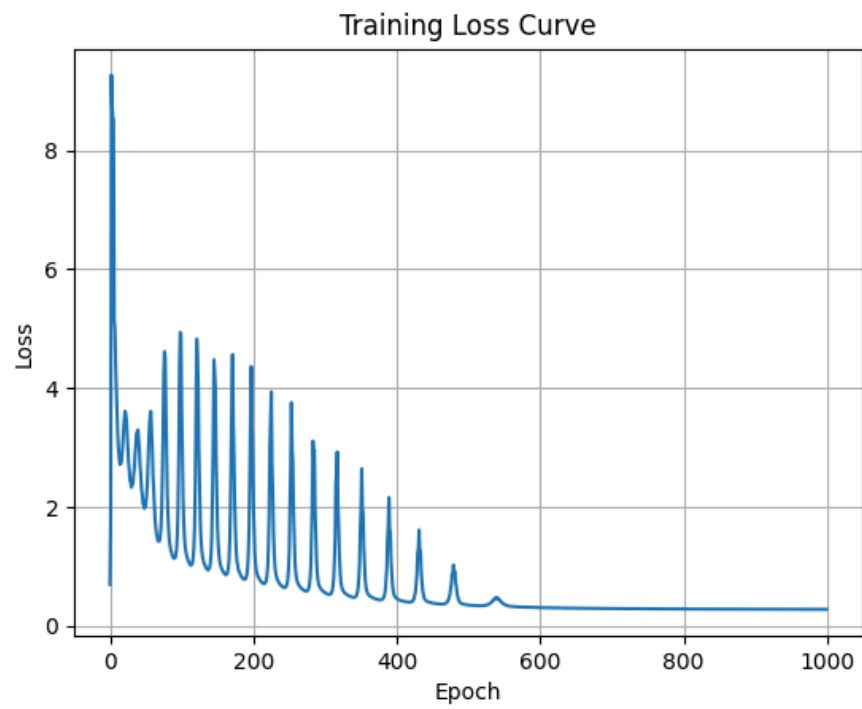
As you can see, the loss also keeps decreasing without divergence.



Use `train_with_validation` to check, there is no overfit problem.

4. Discussion

- I tried to use the exponential decay learning rate in this hw, and the result of the loss looked like this, and I don't know if this is a good approach for machine learning?



- I am not sure whether using batch size will have better accuracy if they converge rapidly just like the above result.